Capítulo 12. Aprender a aprender

Este trabajo se ha traducido utilizando IA. Agradecemos tus opiniones y comentarios: translation-feedback@oreilly.com

> *Vive como si fueras a morir mañana. Aprende como si fueras a vivir para siempre.*
>
> —Mahatma Gandhi

Si llevas un tiempo e e en el ámbito tecnológico, sabrás que la tecnología cambia constantemente. Esto significa que tendrás que enfrentarte al reto de aprender cosas nuevas con regularidad. Muchos ingenieros creen que se les paga por escribir código, lo cual es cierto. Sin embargo, también se les paga por *pensar*. Te ganas la vida utilizando tu cerebro, por lo que debes alimentarlo con conocimientos, lo que significa aprender.

Podría decirse que la capacidad de aprender cosas nuevas rápidamente es la habilidad más importante de un ingeniero de éxito. Además, no solo tienes que comprender las cosas nuevas, sino también aplicar lo

que has aprendido a situaciones reales. Muchas personas se benefician del enfoque «ver uno, hacer uno, enseñar uno».

LA PARADOJA DE LO NUEVO Y BRILLANTE

Los ingenieros de software se sienten atraídos por lo «nuevo» como las polillas por la luz, y cuando algo aparece por primera vez, suele ir acompañado de una gran actividad que hace que parezca que te quedarás atrás si no lo adoptas inmediatamente. Sin embargo, en sus inicios, es posible que las tecnologías no estén listas para su lanzamiento, ya que aún no se han pulido los perímetros. Con el tiempo, el entusiasmo se desvanece. Pero es entonces cuando la novedad suele estar más lista para su adopción. No olvides nunca que las nuevas tecnologías son como los autobuses: en 15 minutos llegará otro.

Estudiar a última hora no funciona

Es probable que en algún momento de tu carrera educativa hayas estudiado a última hora para un examen importante. Y puede que hayas tenido éxito, al menos a corto plazo. Pero ¿retuviste la información? Para aprender realmente algo, debes codificar la información: debe ser elaborada, significativa y tener contexto. Las historias son increíblemente eficaces y se han utilizado durante milenios para enseñar y compartir. El aprendizaje suele implicar repetición y, cuando se espacia adecuadamente, aumenta el conocimiento y la retención.

¿Cómo se aplican las historias a tu caso como ingeniero? Una cosa es que alguien te diga que tengas mucho cuidado al usar comandos como `rm`, especialmente con el indicador `-f`. Y otra cosa muy distinta es que alguien comparta la historia de cómo borró inadvertidamente el disco duro de su ordenador portátil mientras trabajaba en un script que incluía `rm -f`.[1]

A pesar de tus mejores esfuerzos, olvidarás cosas, lo cual es en realidad una característica, ¡no un error! Tu cerebro es increíblemente bueno eliminando información que ya no necesitas. Si bien algunos fragmentos pueden no desaparecer nunca,[2] y otros desaparecen demasiado rápido,[3] tu cerebro está «sacando la basura» de forma activa, por así decirlo. La pérdida de información es predecible, pero no es igual para todas las personas ni para todos los datos. Los ordenadores pueden ayudar repitiendo preguntas a intervalos determinados (lo que, junto con una gamificación bastante ingeniosa, es la forma en que algunas personas logran aprender nuevos idiomas). Es probable que algunos de ustedes hayan recurrido a la técnica poco tecnológica, pero muy eficaz, de utilizar tarjetas didácticas.

Si te cuesta trabajo algo, a menudo es mejor dejarlo a un lado por un tiempo en lugar de seguir dándole vueltas a la cabeza. El simple hecho de darte una ducha, salir a caminar o preparar la cena puede distraerte lo suficiente como para dar con la respuesta. Tu cerebro tiene dos modos de procesamiento distintos: el modo R y el modo L. *El modo R* es el aspecto no verbal, de búsqueda y recuperación, de tu cerebro. Funciona en segundo plano y no se puede controlar directamente, lo que da lugar a resultados impredecibles. Cuando estás viendo una película

y no consigues recordar el nombre de un actor, y cinco minutos después te viene a la cabeza, es el modo R en acción.

El *modo L*, por el contrario, es verbal, analítico y lineal, y se centra en la lógica y el cálculo. Al distraer el modo L, puedes liberar el modo R para que haga lo que mejor sabe hacer. Piensa en el problema y luego haz algo rutinario, como darte una ducha o limpiar la casa.

Como ingeniero, puedes utilizar estos dos modos de procesamiento en tu beneficio. Si te encuentras atascado en un problema, es posible que esforzarte más no sea la solución más eficaz. No tengas miedo de dejar el problema a un lado por un momento, distraer el modo L realizando una tarea más mundana (o dando un paseo) y ver qué pasa. La mayoría de las veces, la solución te vendrá a la mente más tarde.

Adquisición de habilidades

> *La mejor inversión que puedes hacer es en ti mismo. Cuanto más aprendas, más ganarás.*
>
> —Warren Buffett, inversor y filántropo estadounidense

Existen varios modelos de adquisición de habilidades que tienen conceptos similares. Un ejemplo que proviene de las artes marciales es *Shu Ha Ri*, o como dijo una vez Bruce Lee: «Aprende el principio, cumple el principio y disuelve el principio». Otra forma de pensar sobre esto proviene de Clark Terry: Imitar, Asimilar, Innovar. En la fase Shu

o Imitar, los alumnos siguen al instructor al pie de la letra. Te centras en duplicar lo que hace el profesor sin preocuparte demasiado por el porqué o la teoría subyacente.

En la etapa Ha, comienzas a diversificarte. Entiendes los conceptos básicos y tienes suficiente memoria muscular para repetir los movimientos fundamentales. Ahora es cuando comienzas a completar la teoría y a buscar otros maestros o fuentes de información, que luego puedes asimilar a tu comprensión del tema. En la etapa Ri, básicamente estás aprendiendo de ti mismo. Tu práctica impulsa el aprendizaje en lugar de seguir el camino trazado por otros.

El*modelo Dreyfus « »* amplía este concepto dividiendo el aprendizaje en cinco etapas distintas (descritas en la figura 12-1) que funcionan de forma progresiva. En la etapa de novato, los alumnos necesitan recetas: haz *exactamente* esto. Si estás aprendiendo un nuevo deporte, esta es la etapa en la que tu entrenador puede mover tu pie unos grados o hacerte repetir un movimiento aparentemente sin fin. En la etapa de principiante avanzado, empiezas a ir más allá de las reglas rígidas; empiezas a experimentar para ver qué pasa cuando cambias el agarre o modificas la postura.

En la etapa de competente, puedes empezar a resolver problemas. Puedes reconocer cuál fue tu error observando el vuelo de la pelota. Si llegas a la etapa de experto, puedes autocorregirte. Puedes ver o sentir el error y corregirlo sin necesidad de que un entrenador te lo señale. Si trabajas en algo durante el tiempo suficiente, entrarás en la etapa de experto, en la que todo se hace por intuición. Como golfista, solo piensas: «Necesito que esta bola corte» y... ¡corta! De hecho, en esta etapa,

probablemente te costaría explicar qué o cómo has logrado algo; simplemente te ha salido bien.

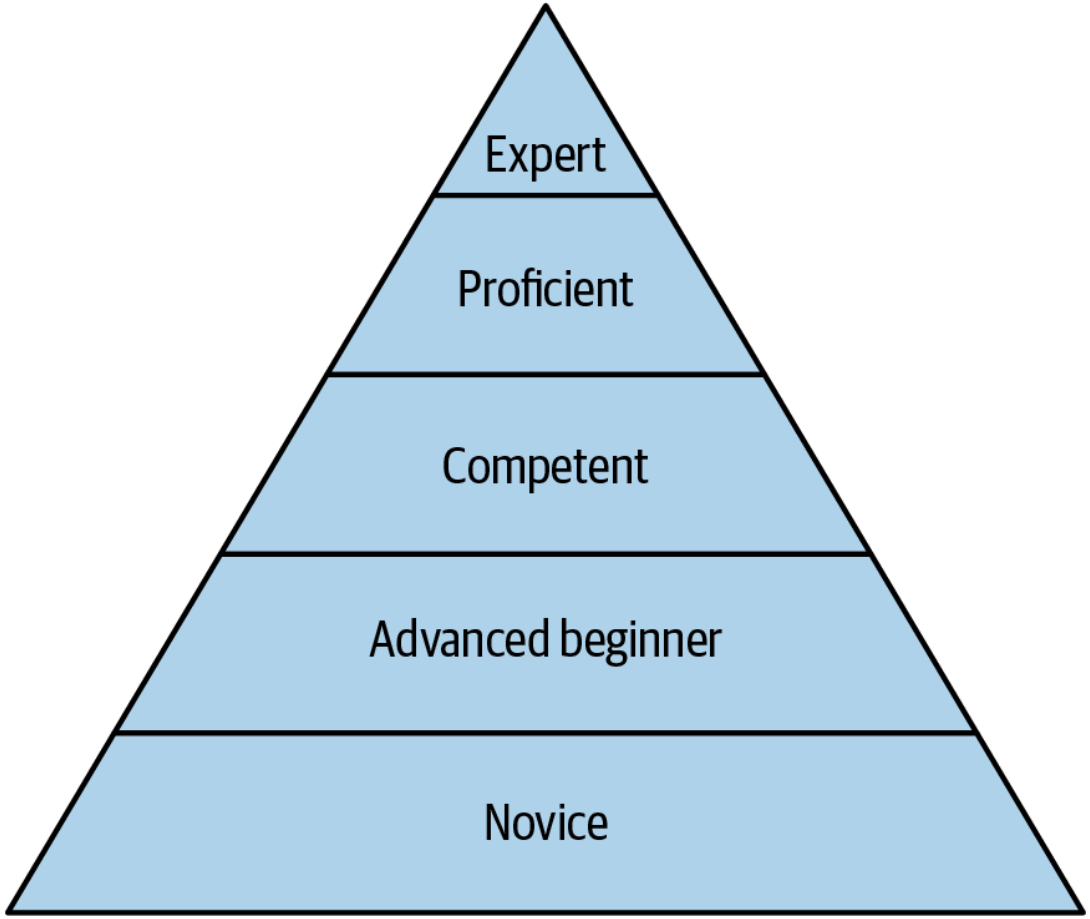


Figura 12-1. El modelo Dreyfus captura el proceso de adquisición de nuevas habilidades

Debe quedar claro que decirle a alguien que acaba de coger un palo de golf que «golpee cortando pensando en golpear cortando» no funcionará muy bien. Cuando se está empezando, se necesitan instrucciones paso a paso, por lo que muchos lenguajes y marcos tienen tutoriales de introducción.[4] Las reglas y las recetas son vitales para los principiantes, pero pueden llegar a ser un obstáculo para los expertos, ya que inhiben su intuición, que tanto les ha costado adquirir. Y al igual que con

la mejora del juego corto, limitarte a ver un vídeo o leer consejos no te llevará muy lejos: ¡tienes que practicar!

Piensa en cómo aprendes un nuevo marco de trabajo. ¿Te lanzas inmediatamente a crear una aplicación compleja? Probablemente no. Empiezas con el proverbial ejemplo «¡Hola, mundo!».[5] Al principio, copias el código directamente de un tutorial y recurres a búsquedas o a un chatbot para solucionar los errores. Sin embargo, con el tiempo, tu intuición se desarrolla y empiezas a reconocer patrones; sabrás dónde está el problema tan pronto como veas el mensaje de error. Después de aún más tiempo, descubrirás dónde están los perímetros del mapa, las partes del marco que cambiarías si pudieras. A medida que te acerques al dominio, ya no traducirás lo que quieres, sino que simplemente expresarás tu intención a través del marco. Cuando se te asigne una tarea, «verás» cómo hacerla.

El hábito de aprender

> *La habilidad más importante que debe tener un programador es la capacidad de aprender cosas nuevas de forma eficaz y eficiente. Vas a estar en constante crecimiento y adquiriendo nuevas tecnologías. La capacidad de hacerlo es más importante que cualquier herramienta o tecnología individual.*
>
> —Ali Spittel, ingeniero de software

Aunque hay pocas reglas estrictas, convertirse en un experto lleva mucho tiempo, por lo que la mayoría de las personas no pasan de ser

principiantes avanzados. Dado el ritmo al que avanza la tecnología, esto tiene consecuencias sorprendentes. La tecnología avanza *rápidamente*; ¿cuánto tiempo le dedicas a algo antes de pasar a la siguiente novedad? Eso no quiere decir que para tener una carrera exitosa sea necesario convertirse en un experto en todos los aspectos del software. A lo largo de tu carrera, tu experiencia crecerá en algunas áreas, mientras que otras habilidades permanecerán en la fase de principiante avanzado, ¡y no hay nada de malo en ello!

El reto es sacar el máximo partido a tu limitado tiempo de aprendizaje. Haz una lista de lo que quieres explorar, priorizando lo que quieres aprender en los próximos meses. Tu lista y tus prioridades serán fluidas; a medida que cambien tus intereses, también lo harán tus objetivos de aprendizaje.

No todo el aprendizaje requiere el mismo nivel de concentración o atención: la profundidad que elijas perseguir cambia en función del tema. Christopher Judd se refiere a este enfoque de manera e e como una estrategia de profundidad de aprendizaje:

Encuesta

> Escucha podcasts, asiste a grupos de usuarios o conferencias, ve las ponencias de conferencias tecnológicas, lee detenidamente las notas de lanzamiento.

Inmersión superficial

> Mira vídeos cortos, lee entradas de blogs y artículos, sigue tutoriales.

Profundiza

Asiste a un taller o curso de formación en vivo, presencial u online, lee libros sobre el tema.

Profundización extrema

Realiza una presentación sobre el tema o dirige un taller o una clase de formación.

El aprendizaje tiene que ser un hábito , ya que los seres humanos olvidamos las cosas. En la década de 1880, Hermann Ebbinghaus estudió la memoria y descubrió que las personas olvidaban la mitad de lo que aprendían tan solo 30 minutos después de aprenderlo. Sin embargo, cuando se expone repetidamente al material, el deterioro se ralentiza considerablemente. Propuso la idea de la repetición espaciada: básicamente, revisar el material a intervalos cada vez mayores después de aprenderlo refuerza los conceptos antes de que se pierdan por completo. Piensa en cómo podrías programar este enfoque a lo largo del año:

Semanal

Escucha podcasts, ve uno o dos vídeos, lee uno o dos artículos.

Mensual

Lee un libro técnico o de negocios, asiste a un grupo de usuarios o a una reunión, realiza un curso en línea.

Anual

Asiste a una conferencia técnica, aprende un nuevo lenguaje de programación, invierte en un proyecto paralelo, obtén una certificación, imparte clases o realiza una presentación técnica.

También puede ser útil «aprender en voz alta». Considera la posibilidad de publicar tu progreso en las redes sociales; a menudo descubrirás una comunidad de personas con ideas afines que te ofrecerán apoyo y ánimo. A medida que descubras cosas nuevas, piensa en crear un vídeo o escribir sobre lo que has aprendido; no te sorprendas cuando una búsqueda futura sobre una pregunta te devuelva algo que tú mismo has creado.[6] Te alegrarás de haberte tomado el tiempo de documentar lo que hiciste.

EL TIOVIVO TECNOLÓGICO

Aquí Nate. Al principio de mi carrera, hablaba con mi entonces jefe y le pregunté por qué habían pasado de la tecnología a la gestión. Me dijeron que estaban cansados de tener que aprender constantemente cosas nuevas, de estar siempre en el carrusel tecnológico; ¡ya habían visto suficientes cosas nuevas! Aunque recién salido de la universidad no apreciaba del todo esa idea, hoy entiendo perfectamente ese sentimiento. ¿Cuántos lenguajes, marcos, bibliotecas frontend o almacenes de datos has aprendido en tu carrera?

Eso no quiere decir que la gestión sea el camino más fácil: hay muchos retos únicos a la hora de dirigir a personas. Pero ser ingeniero de software significa ampliar constantemente tus conocimientos. Puede ser muy estimulante, pero tampoco hay que subestimar lo difícil que puede resultar.

Sabiendo que siempre habrá más que aprender, ¿cómo te mantienes al día? Debes convertirlo en un hábito. Desarrolla una rutina que te funcione. Reserva tiempo en tu calendario para leer y explorar; es posible que tengas que programarlo de vez en cuando. Las tardes de los viernes suelen ser más tranquilas,[7] pero tal vez para ti sea el martes por la mañana o el jueves durante el almuerzo. En última instancia, debes ser el guardián de tu propio tiempo. Algunas personas planifican el aprendizaje de todo un año en un tablero de Trello o establecen recordatorios en el calendario. Experimenta con lo que mejor te funcione, pero hagas lo que hagas, reserva bloques de tiempo específicos para aprender.

Deberías considerar seriamente una práctica que llamamos *«café matutino»* ( ). Dedica los primeros 15-30 minutos de tu día, cuando estés sentado con tu bebida preferida (ya sea un espresso, un té, una bebida energética o cualquier otra cosa), a leer detenidamente las noticias, sea lo que sea que eso signifique para ti. Podría ser pasar un rato en las redes sociales, escuchar un podcast, seguir un tutorial o leer una edición anticipada de un nuevo libro sobre tecnología. Pero conviértelo en un hábito y hazlo antes de que el día se te escape. ¡Seguro que has tenido días que parecen desaparecer en un agujero negro de simulacros de incendio! Dar prioridad a ese tiempo también te ayudará en esos días en los que estuviste tan ocupado que sentiste que no hiciste nada; al menos, ¡aprendiste algo nuevo!

Aprender a través de la IA

La IA puede ser un acelerador del aprendizaje. Aunque no sustituye a la experiencia práctica, el uso de un chatbot puede ayudarte a investigar temas. Los chatbots pueden ser un gran recurso para crear itinerarios de aprendizaje personalizados y profundizar en la tecnología de una manera conversacional y fácil de digerir. Por ejemplo, supongamos que necesitas utilizar una clase de Java con la que no estás familiarizado o que deseas obtener más información sobre un modelo de referencia. Por supuesto, puedes leer la documentación, pero también puedes dejar que la IA lo haga por ti, como se ve en las figuras 12-2 y 12-3.

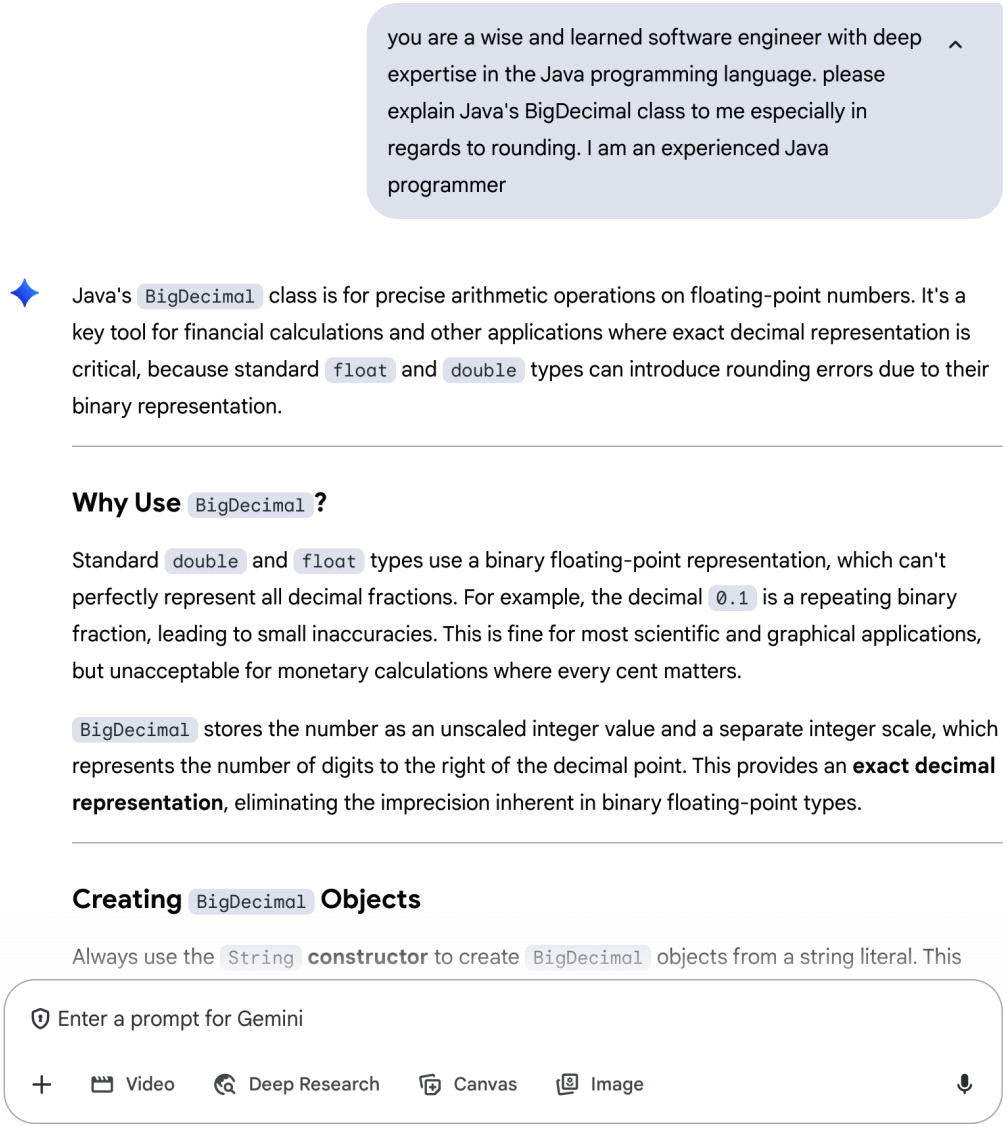


Figura 12-2. Ejemplo de un chatbot explicando una clase

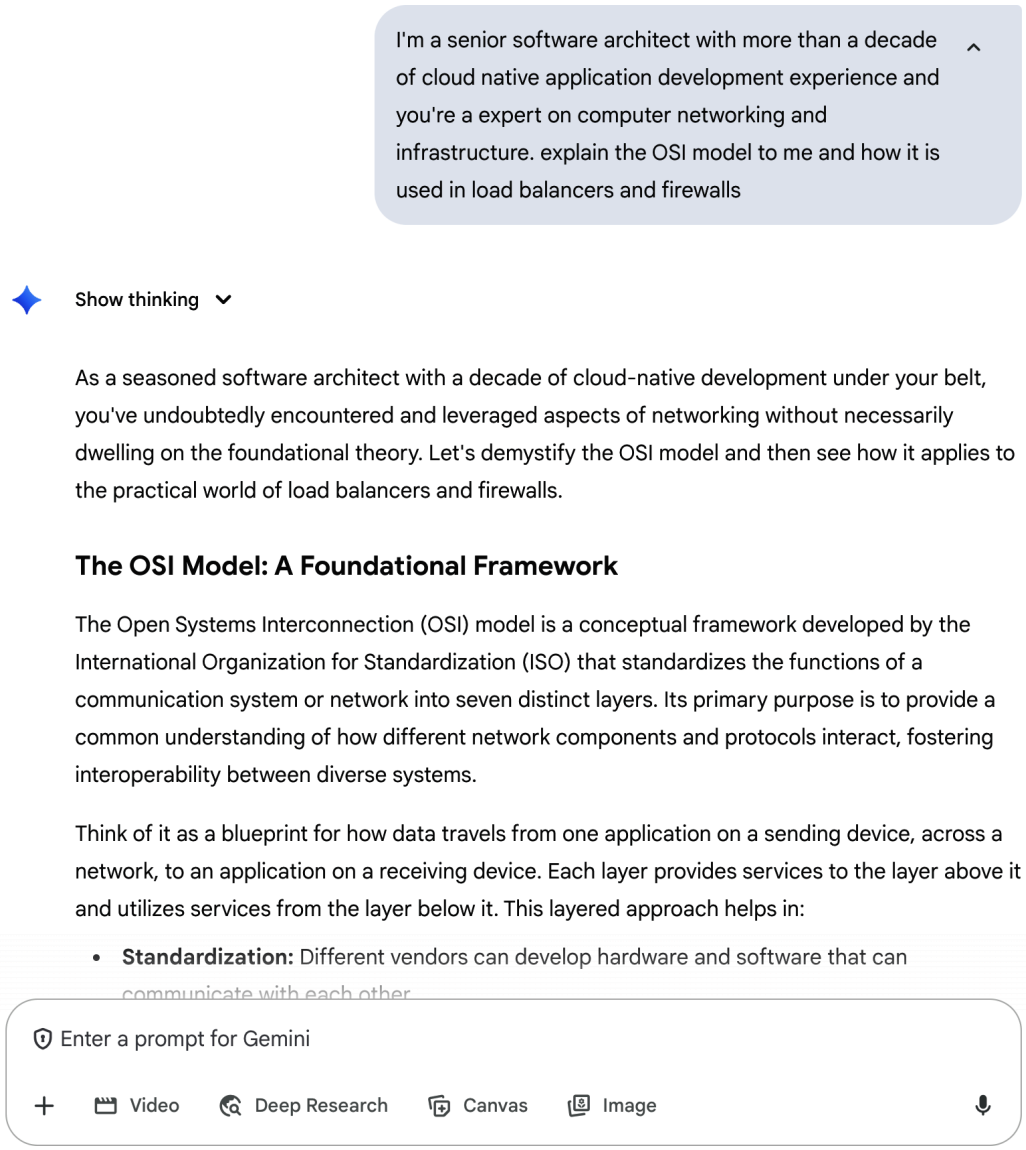


Figura 12-3. Ejemplo de un chatbot explicando el modelo OSI.

Es necesario que seas consciente de las alucinaciones y que compruebes la información con el material original, pero no hay duda de que la IA puede ayudarte a aprender nueva información y a repasar conceptos de tu pasado.

Miedo a perderse algo

Hay muchos datos disponibles: nuevos lenguajes, nuevas técnicas, nuevos enfoques. ¿Cómo mantenerse al día? Tu estilo de aprendizaje preferido dice mucho sobre tu enfoque. Algunas personas prefieren leer artículos y libros, otras prefieren escuchar o ver, mientras que otras aprenden mejor haciendo. No te faltan fuentes de aprendizaje a tu disposición, desde buscar en la web, seguir una lista de lecturas, escuchar podcasts, ver vídeos, asistir a grupos de usuarios y conferencias, y aprovechar recursos del sector como Technology Radar.

La atención es un recurso precioso y no se puede ampliar sin más. No la desperdicies. No puedes leer, ver o escuchar todo; tendrás que ser selectivo. Muchos desarrolladores tienen un miedo legítimo a perderse algo (FOMO, por sus siglas en inglés), ya que si no consumen activamente «todo», se les escapará algo vital.

Lamento desilusionarte, pero es literalmente imposible leer, ver o escuchar todo. De hecho, en el tiempo que te lleva leer este párrafo, ¡se habrá subido a YouTube el equivalente a aproximadamente otra semana de vídeos! Y eso sin tener en cuenta los podcasts, las guías de inicio, las redes sociales o cualquier otro contenido que quieras consumir.

El FOMO tecnológico es real, pero consuélate con la realidad de que, si ocurre algo *realmente* importante en el ámbito tecnológico, te *enterarás*. Puede que te pierdas el anuncio inicial, pero no pasa nada. ¡Hay áreas que no te interesan demasiado, pero de las que aún así te has enterado! No tengas miedo de aprovechar tu red de contactos. Es proba-

ble que tus amigos y compañeros de trabajo tengan intereses diferentes a los tuyos. Aprovecha eso. De hecho, puedes compartir con ellos información interesante que encuentres en tus áreas de especialización, y viceversa. ¡No hay nada de malo en compartir tu aprendizaje con varias personas!

> *Centrarte significa que no podrás surfear todas las olas del océano.*
>
> —Kelsey Hightower, ingeniero de software estadounidense, defensor de los desarrolladores

Tu tiempo y tu atención son recursos increíblemente valiosos.[8] Y son recursos: no puedes simplemente ampliarlos. No puedes permitirte desperdiciar tu atención; debes ser selectivo con cómo la inviertes. No practiques el desarrollo impulsado por el currículum; necesitas una *razón* para aprender algo nuevo. Siempre habrá otra novedad interesante en las redes sociales, y no pasa nada por esperar a aprender algo hasta que lo necesites. Evaluar una nueva tecnología es complicado, pero con la experiencia empezarás a detectar los patrones y verás cómo esa «novedad» es en realidad muy similar a algo de hace unos años. Si tu instinto te dice que algo está sobrevalorado, no hay nada de malo en tomarte un respiro antes de invertir tiempo en ello.

Tu red profesional también puede ser muy útil en este sentido, ya que a veces puedes delegar el aprendizaje a otra persona. Si sabes que un amigo está explorando un ámbito del que tú aún no estás seguro, ponte en contacto con él periódicamente para ver cómo le va. Su experien-

cia puede servirte de guía y, en algunos casos, él resolverá los problemas, lo que te permitirá saltarte algunas dificultades.

> *La atención es un poco como los bienes raíces, en el sentido de que no se puede producir más. Sin embargo, a diferencia de los bienes raíces, su valor sigue aumentando.*
>
> —Seth Godin, autor estadounidense, experto en marketing y empresario

DEJA QUE TU PASIÓN TE GUÍE

Aquí Nate. Una vez intenté aprender más sobre bases de datos, así que un día cualquiera me senté y empecé a leer un libro sobre el tema. Después de unos 20 minutos, dejé el libro, subí las escaleras y me puse a ordenar mi cajón de los calcetines. No es un eufemismo: decidí que mi cajón de los calcetines era más importante. Esa fue una señal inequívoca de que, de hecho, no me apasionaba el tema concreto de las bases de datos. Aunque puedes programar fácilmente un tiempo para aprender, sigue quedando una pregunta bastante importante: ¿qué deberías estudiar? En última instancia, eso depende de ti, pero debes dejar que tu pasión te guíe. Si un tema no te entusiasma, te resultará difícil encontrar tiempo en tu día a día para practicar.

Elige algo que te motive y profundiza en ello. Sigue a los expertos, escucha los podcasts, mira los vídeos y lee la documentación. Lee «Encuentra lo que te apasiona». Ten en cuenta que puedes cambiar a algo diferente en cualquier momento. No hay nada de malo en dejar un libro, cerrar una pestaña o dejar un vídeo a medias; no se ganan puntos extra por terminarlo. Si sigues entusiasmado con un tema determinado, ¡genial, sigue adelante! Si ya no te hace feliz, fantástico, elige algo nuevo. Repite el proceso.

Del mismo modo que no puedes estar al día de todas las novedades en materia de software, tampoco puedes adoptar todas las novedades. Tu pasión por una herramienta o tecnología determinada no significa automáticamente que sea adecuada para tu proyecto u organización. Muchos desarrolladores han practicado, en algún momento u otro, el

desarrollo impulsado por el currículum. En otras palabras, adoptaron una nueva tecnología porque querían añadirla a su currículum, no porque fuera a beneficiar significativamente al proyecto.

Debes ser un consumidor crítico de las novedades. El software es un conjunto constante de compensaciones y, a pesar de lo que se suele enseñar, rara vez existe una forma «óptima» de hacerlo.[9] No te centres solo en los aspectos positivos de una tecnología, sino que ten en cuenta todas las consecuencias. Sé un consumidor informado y recuerda las sabias palabras de Rich Hickey: «Los programadores conocen las ventajas de todo y las compensaciones de nada».

¿Dónde debes invertir tu tiempo?

La intuición crece con la experiencia y, con el paso de los años, empezarás a tener una idea de qué cosas tienen poder de permanencia y cuáles son probablemente solo un éxito efímero. En última instancia, verás hacia dónde llevan las tendencias a la industria. Sin embargo, debes moderar un poco tus corazonadas. Paul Graham escribió una vez sobre el radar del hacker, describiendo algunos de los criterios que tiene en cuenta al analizar una nueva tecnología.

Después de aplicar ese radar hacker a Java, llegó a la conclusión de que «tengo la corazonada de que [Java] no será un lenguaje muy exitoso». Ahora bien, hizo esta declaración en 2001, cuando Java era bastante nuevo. Más de 20 años después, con la perspectiva que da el tiempo, Java es obviamente uno de los lenguajes más exitosos de todos los tiempos. Si alguien tan experimentado y exitoso como Graham puede

equivocarse tanto, quizá sea conveniente que moderes un poco tu propia intuición.

Dicho esto, juzgar las apariencias puede ser útil. En el software, los patrones suelen repetirse y las cosas tienden a oscilar como un péndulo entre extremos. Es habitual que se desempolven cosas antiguas, se refinen, se renombren y se vuelvan a presentar como si nadie lo hubiera hecho antes. Considera esta descripción:

> «Un enfoque que permite que los componentes de software escritos en diferentes lenguajes de programación y que se ejecutan en varios ordenadores se comuniquen y funcionen juntos».

¿Qué tecnología crees que describe esa definición? ¿Estás pensando en los microservicios? ¡Encaja! Sin embargo, esa es una definición de la Arquitectura Común de Intermediación de Solicitudes de Objetos (COBRA), que fue popular a mediados y finales de la década de 1990, muchos años antes de que nadie hubiera siquiera contemplado una arquitectura de microservicios.

Con la experiencia, empezarás a ver cómo una nueva tecnología es igual que otra que utilizabas hace unos años.[10] Quizás los problemas con esa tecnología se hayan resuelto, o quizás no. Debes hacer preguntas incisivas. El hecho de que algo sea nuevo no significa que vaya a beneficiar a tus proyectos.

La sabiduría colectiva *puede* ser una guía útil. ¿Qué es lo que está captando la atención de la comunidad? Eso puede llevarte a cosas inter-

esantes. Pero a veces las masas se dejan llevar por el entusiasmo. Una vez más, aplica también tus propias experiencias.

Puedes aprovechar los recursos del sector, como el Technology Radar de Thoughtworks. Actualizado dos veces al año, el Technology Radar es una instantánea de herramientas, técnicas, plataformas, lenguajes y marcos. El radar asimila experiencias de múltiples proyectos para generar un análisis de la tecnología lo más objetivo posible. Seguir el Radar puede introducirte en cosas nuevas, además de darte una idea de en qué punto del ciclo de adopción se encuentran esas tecnologías.

CREA TU PROPIO RADAR TECNOLÓGICO

Con suerte, tu empresa ofrece un amplio abanico de oportunidades de aprendizaje que puedes aprovechar. Sin embargo, como ingeniero de software, tú eres el responsable último de tu trayectoria profesional. Con un sinfín de nuevos lenguajes y tecnologías, puede resultar increíblemente difícil decidir cómo distribuir tu valiosa atención. Es muy fácil quedarse atascado en el análisis paralizante, algo que puedes evitar creando tu propio radar tecnológico.

No tiene por qué ser excesivamente complejo ni tan impresionante gráficamente como el producto de Thoughtworks. Pero adquirir el hábito de revisar periódicamente lo que *te* parece interesante y emocionante puede marcar la diferencia en tu aprendizaje. Puedes leer más sobre cómo hacerlo en el capítulo 14.

Practica la innovación

A medida que aprendas y explores, encontrarás algo que sin duda beneficiará a tu proyecto, tu equipo o tu organización. Incorporar esas ideas a tu organización puede darte una mayor visibilidad en tu empresa, lo que te ayudará a conseguir aumentos de sueldo y ascensos. Pero, ¿cómo se introducen las novedades?

Un club de lectura puede ser una de las formas más sencillas y eficaces de introducir nuevas tecnologías. Busca un grupo de ingenieros con ideas afines, elige un libro técnico y ponte manos a la obra. Intenta reunirte semanalmente, quizá a primera hora de la mañana o durante la hora del almuerzo. Casi todas las organizaciones cubren el coste de los libros y muchas incluso están dispuestas a encargarse del catering para el almuerzo o el desayuno. Los clubes de lectura también sirven como motivadores: es mucho más fácil leer un capítulo cuando sabes que lo vas a comentar con tu equipo el martes.

Muchas organizaciones e es practican alguna variante del *día de la innovación*, a menudo llamado *«hackers days»* o *«hackathons»*. Independientemente del término, el concepto es el mismo: proporcionar a las personas un espacio seguro y práctico para explorar nuevas tecnologías. Los «hackers days» actúan como una válvula de escape para el inevitable deseo de jugar con las nuevas tecnologías. Sin ellos, muchos desarrolladores intentarían introducir a escondidas esa nueva biblioteca o lenguaje en un proyecto, apostando por la utilidad de pedir perdón después en lugar de pedir permiso.

Si tu empresa no tiene un concepto recurrente de hacker day, ¡introdúcelo! Preséntalo como un experimento y empieza poco a poco, tal vez solo un viernes por la tarde. Involucra a tantas partes de tu organización como puedas; te sorprenderá descubrir dónde se esconden algunas ideas revolucionarias. Aunque es posible que el código resultante nunca llegue a producirse, aprenderás algo. Quizás un nuevo marco parecía realmente prometedor, pero cuando entró en contacto con tu contexto particular, te diste cuenta rápidamente de que requeriría una reescritura casi completa. Quizás ese nuevo lenguaje resuelve con elegancia un problema complicado con el que llevas semanas luchando. No lo sabrás hasta que lo pruebes.

Las presentaciones periódicas de proyectos, ya sean a nivel de departamento o de empresa, son una forma excelente de conocer las tecnologías que se utilizan en tu organización. La mecánica varía desde demostraciones breves, de la duración de una charla relámpago, hasta un conjunto de stands en los que diferentes grupos presentan su trabajo. Las jornadas de demostración no solo son una forma estupenda de conocer las tecnologías que se utilizan en la empresa, sino también una oportunidad para identificar a las personas con las que puedes hablar cuando surge una necesidad en tu proyecto.

Los hackatones no son la única opción de innovación. Las series de charlas tecnológicas periódicas son una forma inestimable de introducir nuevos conceptos en una empresa. Tanto si los ponentes son compañeros de trabajo como expertos del sector, ofrecer sesiones semanales de almuerzo y aprendizaje es una de las formas más sencillas de mantenerse al día con el cambiante panorama tecnológico.[11] Una vez más, si tu empresa no cuenta con una serie de charlas, ¡crea una! Ofre-

cerse como voluntario para dar una charla sobre una tecnología que estás tratando de aprender es una poderosa fuente de motivación.

Las sesiones de almuerzo y aprendizaje no tienen por qué ser para toda la empresa para ser eficaces. Los enfoques más localizados también son excelentes. ¿Has leído un artículo interesante o has visto un vídeo revelador? Compártelo con tu equipo. ¿Buscas una motivación extra para leer un libro nuevo? Crea un club de lectura con algunas personas afines e turnaos para dirigir el debate. Si te entusiasman las nuevas tecnologías, ¡organiza una reunión quincenal para debatir sobre ellas!

La mayoría de las áreas metropolitanas de tamaño considerable tienen un calendario regular de encuentros locales y reuniones de grupos de usuarios. Es probable que encuentres una charla tecnológica interesante casi todas las tardes (y es probable que te inviten a cenar o, al menos, a tomar algo).[12] Aunque los grandes eventos del sector en Las Vegas o San Francisco pueden atraer más la atención de la prensa tecnológica, hay muchos eventos regionales excelentes que no te obligarán a gastar miles de dólares en viajes.

Si ya tienes una serie de almuerzos formativos con buena asistencia y has asistido a un evento regional, podrías dar un paso más y organizar una conferencia interna. Estas pueden ir desde eventos informales de medio día con un puñado de ponentes internos hasta eventos completos de varios días y en varios lugares, en los que participan expertos del sector ajenos a tu organización.[13] No subestimes el esfuerzo que supone organizar y llevar a cabo incluso pequeños eventos internos. Pero la recompensa puede ser enorme.

Los eventos internos unen a las personas y ayudan a difundir nuevas ideas. Ofrecen la oportunidad de interactuar con personas con las que normalmente no se relacionan en el día a día. También son una excelente oportunidad para poner a prueba tus habilidades de presentación. Aunque presentar provoca un miedo primario en muchas personas, es un excelente motivador si estás tratando de aprender algo nuevo. Por no hablar de que la visibilidad puede acelerar tu crecimiento profesional, preparándote para un ascenso o una mayor responsabilidad en los proyectos.

Reuniones informativas sobre arquitectura

Algunas organizaciones tienen un enfoque formalizado que utiliza sesiones informativas sobre arquitectura. Básicamente, una persona investiga y presenta los resultados al equipo. Y no, no es necesario ser arquitecto para impartir una sesión informativa sobre arquitectura. Aunque hay muchas preguntas que puedes explorar, debes asegurarte de abordar las siguientes:

- ¿Por qué deberías utilizar esta tecnología? ¿Por qué *no* deberías utilizar esta tecnología?
- ¿Qué necesitas saber para responder a la pregunta anterior?
- ¿Qué necesitas saber para utilizar esta tecnología?

La presentación resultante no tiene por qué durar varias horas; 45-60 minutos son más que suficientes. No se trata de una guía detallada,

pero debe ir más allá de la documentación básica de introducción. Estas sesiones deben ser interactivas y fomentar la participación bidireccional. Los asistentes deben tomar notas y hacer buenas preguntas, aportando sus propias experiencias al tema.

Si superas la fase informativa, es hora de arremangarse y ponerse manos a la obra. Es hora de un taller. Por desgracia, nunca tendrás tiempo suficiente para realizar un taller. ¿Estás tratando de decidir entre dos tecnologías frontend que compiten entre sí? Hay una forma infalible de tomar esa decisión: crea la aplicación dos veces, una con cada una, y descarta la que no te guste. Buena suerte para que te lo aprueben.[14] Tendrás que determinar las cosas más importantes que debes demostrar con tu tiempo limitado.

Céntrate en explorar las características clave y prepara una configuración sencilla para los participantes. Expresa claramente tus objetivos y ofrece material de seguimiento para aquellos que deseen profundizar después de la sesión. No tengas miedo de probar cosas y curiosear por todos los rincones; puede que te sorprenda gratamente (o desagradablemente) lo que descubras. ¿Qué funciona? ¿Qué no funciona? ¿Qué pasa cuando te sales del camino marcado?

Asegúrate de que tu taller esté sujeto a las limitaciones que la tecnología encontrará en el proyecto objetivo. Si vas a añadir esto a un proyecto heredado existente, prueba la tecnología con esas limitaciones heredadas. Conseguir que algo funcione en un entorno nuevo puede ser útil, pero puede dejarte con una imagen incompleta.

LIMITACIONES DE LOS PROYECTOS HEREDADOS

Aquí Nate. Hace años, un participante en uno de mis talleres compartió su experiencia al intentar utilizar una nueva tecnología de interfaz de usuario en su proyecto. Se habían topado con un obstáculo con la biblioteca que habían elegido años atrás y necesitaban hacer un cambio. El equipo estudió el campo y redujo las opciones a tres. Dividieron las bibliotecas y realizaron algunos talleres desde cero. Finalmente, se decidieron por una opción que planeaban incorporar a su aplicación.

Seis meses después, se dieron cuenta de que esta biblioteca no se podía añadir aquí y allá; para utilizarla realmente, tendrían que refactorizar completamente su aplicación. Admitieron que se habían dejado llevar por el entusiasmo que rodeaba a la biblioteca que habían elegido, pero también está claro que deberían haber realizado su explotación utilizando una bifurcación de su proyecto existente. Probablemente habrían descubierto mucho antes la necesidad de una mayor refactorización. Ahora bien, es posible que hubieras elegido igualmente esa biblioteca, pero al menos lo habrías hecho con los ojos bien abiertos.

Moraleja de la historia: el hecho de que puedas hacer que la lista de tareas pendientes funcione desde cero no significa que una tecnología determinada vaya a sobrevivir al contacto con tu entorno. Juegas como entrenas; no pruebes cosas nuevas en el vacío.

Una vez que hayas tenido tiempo para practicar, podrás tomar decisiones informadas sobre una tecnología. Si todo sale bien, es el momento de buscar un proyecto de prueba en tu organización. Asegúrate de que

el proyecto de prueba sea adecuado y de que un fracaso no condene al fracaso un proyecto estratégico crítico.

Practica la elegancia

Mantenerse al día con el panorama tecnológico en rápida evolución es importante y, francamente, una parte vital de tus habilidades de ingeniería de software. Sin embargo, puede resultar muy abrumador. Sé amable contigo mismo; practica la paciencia. Es muy fácil agotarse al intentar mantenerse al día en un sector tan diverso como el del software. Aprender debe ser divertido, no un motivo de estrés en tu vida. No dejes que te quite el sueño o te afecte a tu salud mental (o física).

No hay nada de malo en esperar hasta el último momento responsable para aprender algo. La tecnología está llena de ciclos de exageración; no tengas miedo de dejar que algo se desarrolle antes de dedicarle tu tiempo.[15] La paciencia es una virtud; el mundo del software está plagado de restos abandonados de lo que estaba de moda el mes pasado. No hay un plazo límite para aprender algo: si algo demuestra que tiene poder de permanencia, siempre puedes aprenderlo la semana que viene, el mes que viene o el año que viene.

Conclusión

La tecnología cambia. Rápidamente. A lo largo de tu carrera, trabajarás con una amplia variedad de herramientas, lenguajes y tecnologías; tu capacidad para aprender, adaptarte y ajustarte es vital para tu éxi-

to. Mantener tus habilidades al día debe ser un hábito, así que reserva tiempo en tu agenda para probar cosas nuevas. Una cadencia regular es más sostenible que concentrar el aprendizaje de todo un año en un fin de semana largo. ¡Disfruta del proceso!

Ponlo en práctica

En última instancia, para aprender una nueva tecnología, tienes que ponerla en práctica literalmente. Considera la posibilidad de crear una aplicación personal para satisfacer una necesidad; tal vez quieras gestionar tu creciente colección de vinos o crear una aplicación para llevar un registro de las estadísticas deportivas de tu hijo. Si no tienes ninguna necesidad urgente, ponte en contacto con tu escuela local o con organizaciones sin ánimo de lucro, ya que a menudo buscan ayuda tecnológica y suelen darte mucha libertad para probar cosas nuevas.

Tómate un momento para estudiar el campo de la tecnología: ¿qué es lo que más interés genera hoy en día? ¿Te llama la atención? Si no es así, ¡elige algo que sí lo haga! Profundiza en lo que te entusiasma, sigue a los líderes de opinión en las redes sociales y suscríbete a podcasts, transmisiones, boletines informativos y vídeos. ¿Hay algún grupo de usuarios local al que puedas asistir? ¿Hay alguna conferencia centrada en ello? ¿Y eventos virtuales? Sumérgete en ese espacio. Intenta crear un proyecto personal utilizando esa tecnología. Haz una presentación sobre él a tus colegas, en tu grupo de usuarios local o en una conferencia.

Cada mes o dos, reevalúa la situación. ¿Sigues sintiendo pasión por esa tecnología? Si no es así, ¡elige algo nuevo! Repite el proceso. No tengas miedo de volver a algo que hayas explorado anteriormente; no tendrás que volver a aprenderlo todo y probablemente ampliarás tus conocimientos.

Al final, nada es tan eficaz como dedicar unos días o semanas a crear código funcional. No tiene por qué ser elegante, no tiene por qué cubrir todos los perímetros posibles y puede ser suficiente para demostrar la viabilidad de la tecnología. No hay atajos para aprender, no hay forma de «cargarse la información en la cabeza».

Aprender en voz alta también puede ser una forma eficaz de enseñar. No tengas miedo de unir fuerzas con un par de amigos o compañeros de trabajo para llevar a cabo un pequeño proyecto juntos. Incluso un solo par de ojos adicional puede ser de ayuda, por lo que trabajar en pareja es un enfoque eficaz. Considera la posibilidad de convertir tu proyecto en código abierto o, al menos, alojarlo en un repositorio público para que otros puedan comentar y contribuir. También puedes considerar la posibilidad de retransmitir en directo tu aprendizaje; aunque puede resultar intimidante, el efecto comunitario es poderoso. Es probable que otras personas se hayan encontrado con problemas similares y suelen ofrecer sus consejos. El simple hecho de darte cuenta de que no estás solo en tu viaje de aprendizaje puede ser increíblemente reconfortante.

Recursos adicionales

- «Thinking Architecturally» (informe) de Nathaniel Schutta (O'Reilly, 2018)
- *The Pragmatic Programmer*, de Andrew Hunt y David Thomas (Addison-Wesley Professional, 1999)
- *The Passionate Programmer*, de Chad Fowler (Pragmatic Bookshelf, 2009)
- Radar tecnológico de Thoughtworks
- *Las primeras 20 horas: cómo aprender cualquier cosa... ¡rápido!* por Josh Kaufman (Portfolio, 2014)
- *Pensamiento y aprendizaje pragmáticos*, de Andy Hunt (Pragmatic Bookshelf, 2008).

1 Alguien con quien tus autores pueden o no tener experiencia directa.

2 Por ejemplo, un número de teléfono de tu juventud o el nombre de tu profesor favorito de la escuela primaria.

3 ¿Cómo dijiste que te llamabas?

4 Además, ¿quién no necesita otra lista de tareas pendientes, verdad?

5 ¿Cuántas aplicaciones de listas de tareas pendientes has creado?

6 Ambos autores pueden dar fe de haber encontrado material propio que habían olvidado por completo que habían creado.

[7] Cualquiera que programe una reunión un viernes por la tarde ha cometido un acto hostil.

[8] El tiempo es un recurso no renovable. Úsalo con prudencia.

[9] La mayoría de las veces, eliges el enfoque menos malo.

[10] O una tecnología creada inicialmente para un fin puede resurgir más tarde para resolver un problema diferente. Por ejemplo, Internet se reutilizó para uso civil después de haber sido desarrollado inicialmente por el ejército.

[11] O, como lo denominó una organización, «masticar y escupir». No es nuestro apodo preferido, que conste.

[12] Ofrecerse como voluntario para dar una charla en un evento local es una excelente forma de iniciarse en las conferencias, si es que te interesa ese camino.

[13] Los autores de este artículo han participado en *muchos* de estos eventos para nuestros clientes; ponte en contacto con nosotros si podemos ayudarte.

[14] Si alguna vez lo consigues, envía una nota a los autores; nos encantaría saber más.

[15] El tiempo es un recurso no renovable. Úsalo con prudencia.